

Tooling limitations and repository audit-readiness recommendations

Read-only audit · October 5, 2026 · byronwall/explorEDA

Revision: 362db58082df9c1b57c2305a49823a1dae385081

Fresh application browser tests: 0 completed; navigation blocked by administrator policy.

Summary of the tooling boundary

Repository read access worked. Actual application-browser verification did not. The decisive restriction was a managed Chromium policy that blocks every URL, including the public application, local HTTP and file navigation. No repository modification can remove that session policy. The appropriate solution is an authorized browser/CI execution environment, not a workaround that disables security controls.

The repository nevertheless has several fixable audit-readiness gaps: the coverage matrix is development-only without a retained matching review build; historical screenshot links point into absent tmp-root files; the deployment artifact has roughly one-day retention; source/intent/proof records are not generated from a shared schema; and reviewers must reconcile mixed-age documents by hand. These are separate from the environment restriction. [HOST REVIEW TMP DEPLOY BASE INV](#)

Difficulty register

Difficulty actually encountered	Layer / impact	Recovery used or final limit
Container could not resolve external hosts, including raw GitHub	Execution network; prevented ordinary download/clone/build-source acquisition	Used connected GitHub reads instead; did not infer that the sites were down
Web fetch could not open the public demo or codeload archive	Web transport; no reachable live review target	Retained as access failure, not product availability evidence
Generic archive download rejected a URL not available through its fetch path	Download-tool boundary	Used the GitHub Actions artifact download action successfully
Repository-wide actions/artifacts endpoint was not accepted by the connector	Connector endpoint allowlist	Located a concrete deployment run, then its run-specific artifact list
Long GitHub file responses were truncated in their initial display	Read transport; risk of missing transcript or report passages	Expanded prior response resources in ranges until every copied transcript and key report was read
Files materialization rejected GitHub text-result IDs as not visible in that file surface	Cross-tool file-reference mismatch	Read the connector response resources directly; did not invent sandbox paths or claim a local source checkout
Streaming shell sessions were unsupported in the container interface	Process-control boundary	Started local server/browser with background processes and used their logs/CDP endpoint

Difficulty actually encountered	Layer / impact	Recovery used or final limit
Public, local HTTP and file navigation all failed with administrator blocking	Managed browser environment; prevented all app-level tests	Captured three real block pages and structured errors; stopped without policy changes
Browser alternative was discoverable but not installed/connected	External service boundary	TinyFish suggestion surfaced; no task was executed through it and no success was assumed
A first error-page capture raced a changing execution context	Evidence capture timing	Waited briefly for the committed error page and captured final screenshots/text
Coverage route was absent from the production artifact	Intentional repository build boundary, not a bug	Source inspection found the DEV-only import; documented that BT35 needs a development target
Historical review screenshot paths were not retained at their referenced tmp-root locations	Repository evidence retention	Preserved narrative as historical evidence only; independently checked numerical oracles from deployed data
Downloaded build had no observed source maps/source snapshot	Artifact scope; limited automated source-to-bundle analysis	Continued targeted pinned source reads; did not pretend the minified bundle was a complete semantic audit
Local runtime differs from documented Node 24/pnpm 11.9 setup	Potential reproducibility boundary	No local full-source build/test attempt was made; this is not reported as a failed pnpm check

An optional Markdown conversion module was unavailable during report production; the installed Pandoc and WeasyPrint pipeline produced the reports instead. PDF rendering and layout checks are document-production checks, not application-browser tests.

The browser harness also emitted a Node URL-parser deprecation warning from tooling. It was not an explorEDA browser console warning. No conclusions about the app's console cleanliness follow because the app never started.

What worked well

The connected repository API supported pinned source reads, search, pull-request metadata, deployment metadata and binary artifact download without write access. The transcript archive has a readable inventory and original-file provenance. The chart registry is explicit. Coverage tests already keep registered chart types and example IDs synchronized with the catalogue. The current closure clearly records waived checks rather than claiming they passed. Local datasets and geometry-oriented chart contracts reduce reliance on external data services. Preserve these strengths. [ARCHIVE REG](#)
[COVTEST CLOSE CHART](#)

Recommended improvements, in practical order

R1 · Provide an authorized, deterministic browser review target

Publish a commit-specific application preview or make the build artifact runnable in a CI environment where browser navigation is allowed. Provide one documented base URL and one command to serve the exact bundle, with a health check and explicit desktop support policy. Include the tested SHA in a machine-readable build metadata file and, preferably, in a small visible diagnostics panel.

A separate development coverage target should be explicit: a dev artifact, internal preview, or static generated coverage page. Do not change production behavior merely to compensate for a review tool assuming the wrong route. Keep review-only functionality gated deliberately. [HOST](#)

Acceptance: an authorized browser loads the landing page and a deterministic example, reads its build SHA, and collects a screenshot/trace without secret credentials or ad hoc route discovery. BT35 targets the documented development matrix route rather than production.

R2 · Retain browser proof with revision and data identity

Store screenshots, browser traces, console records, downloads and small observed-result JSON as workflow artifacts or committed synthetic evidence, with stable links. Extend retention beyond a single day for evidence referenced by permanent documentation. Avoid making a long-lived report depend on `tmp/*.png` files that are not retained. The current deployment artifact was saved in this ZIP so a later authorized run can still test the same build. [REVIEW TMP DEPLOY](#)

Each record should include tested source/build SHA, dataset SHA-256, browser/version, viewport, locale/timezone, test ID, steps, expected/observed source keys and metric values, and artifact digest/location. Accessibility proof should identify the actual assistive technology and task, not merely list accessible names.

Acceptance: every reviewed manifest assignment resolves to a specific proof record and artifact. A broken link fails a validation check. Historical proof remains historical after source changes; its applicability is separately computed.

R3 · Generate the audit, matrix and outcome ledger from shared records

Add a small machine-readable requirement catalogue with stable IDs, transcript references, intent strength, scope, implementation sources and verification records. Generate human-readable gap and coverage reports from it. Retain original transcripts unedited, and keep implementation decisions separate from paraphrased intent.

The existing coverage tests already compare registry and example lists. Extend them rather than replacing them: validate report anchors, artifact existence, valid dates, explicit partial/exception states, duplicate IDs and stale proof dependencies. Distinguish declared example usage from observed behavior. [COV](#) [COVTEST](#)

Acceptance: a newly registered family creates a required catalogue entry; a new outcome or scope exception is traceable to intent; a code change that affects a proof dependency is shown as needing review; generated summaries cannot disagree about eleven versus eighteen view types.

R4 · Archive old audits instead of patching contradictory current claims into them

The old analysis is valuable as a dated record but mixes old baseline headers, repair records, new implementation passages and stale next steps. The feature inventory explicitly admits a partial re-audit while retaining older all-eleven/test-total wording. Freeze dated snapshots and maintain a clear latest index with per-section revision/proof dates. [BASE INV](#)

A retirement entry should continue to record the approved scope and waived work. A later audit can recommend a new verification batch without silently reopening the old initiative or changing a waiver into a pass. Likewise, a merged planning document should remain planned-only. [CLOSE PLAN](#)

Acceptance: a reader can determine the latest implementation snapshot, the latest applicable browser proof and the approved future scope without reconstructing a merge history or reading every ticket.

R5 · Provide a compact read-only audit input artifact

Alongside a production build, publish an owner-approved source snapshot for review: relevant source, tests, package/lock files, transcript inventory, active requirements, synthetic datasets and build metadata. Include source maps as restricted review artifacts when useful. This is especially helpful when a connector can read text and download Actions artifacts but cannot provide a repository archive through general networking.

Use only synthetic/public data and exclude secrets, user uploads, browser profiles and environment credentials. Do not bundle a developer's whole working directory. A small manifest of file paths, sizes and hashes is enough to make the snapshot auditable. Keep external fonts optional through fallbacks and avoid critical runtime data/geometry fetches outside the local review bundle. [DEPLOY README](#)

Acceptance: the artifact can be unpacked and analyzed offline with a complete source/fixture manifest, even after the original workflow's short-lived artifact URL expires. A reviewer can reproduce the documented build under the pinned toolchain in an authorized environment.

R6 · Add small known-answer browser fixtures to the repository workflow

Use the included six-row edge fixture and typed-category fixture as proposals, not files already added to the repo. Add deterministic browser scenarios for source versus eligible contributors, date offsets/leap days, typed categories, pair selection, missing/zero/invalid results, shared calculations, JSON restore and actual CSV downloads. Run larger examples separately for performance and layout.

Give important controls stable accessible names and, where needed, stable test IDs. A read-only diagnostic export of canonical settings, selected source keys, eligible inputs and transform outputs can make tests much less dependent on pixel reading. Do not rely solely on internal state exports: tests still need to actuate real controls and verify the visible response.

Acceptance: a browser test selects a mark, inspects its contributors and verifies the exact source keys/value; a restore test compares pre/post applied state and source rows; a CSV test parses actual downloaded bytes. Proof belongs to the exact build, not just a reducer helper.

R7 · Measure before changing performance architecture

Expose repeatable data-shape sweeps for rows, columns, charts and facets, with load/filter/Apply/scroll/heap measurements and hardware/browser metadata. The existing 10,000-row examples are useful starting points, not a declared capacity guarantee. Shared numerical functions reduce semantic divergence but do not prove shared execution or sufficient responsiveness. [INV TABLE ROLL](#)

Acceptance: a report shows measured distributions and known bottlenecks. Any worker, cache, projection or rendering optimization is justified by observed cost and preserves source/selection correctness.

Minimal future audit contract

A single `audit-index.json` could point to: the immutable source revision; current feature/outcome manifest; transcript inventory; supported screen policy; exact application and development coverage targets; small fixture paths/hashes; commands for checks and browser tests; and retained proof/waiver records. Generated `audit-summary.json` should expose implementation counts, review freshness and unresolved outcomes without a misleading global completion percentage.

A review action should require only read access to code and artifacts. No issue creation, branch push or document edit should be necessary to produce an audit. This session followed that boundary: every repository operation was a read, and all created artifacts live only in the conversation's output directory.

What remains uncompleted because of tools

Fresh explorEDA rendering, interaction, browser downloads, saved-state roundtrips, accessibility task execution, performance measurement and a live coverage-matrix walkthrough were not completed. The included plans do not substitute for those tests. The user-visible value delivered is the complete transcript reinterpretation, current source/doc reconciliation, coverage recount/commentary, retained exact build, reproducible data oracles and an honest path to decisive verification.

Source labels link to the identified repository files or PR/run records. Full source locators and reading scope are in Report 06.